

Data Science Lab in R - Lab Two

Ishanvi Verma

2025-06-22

Short Answer Questions

For the following questions, please response in 3-4 sentences.

Question 1

Many people expect a coin flip to be 50-50 (i.e. a binomial distribution). Read this article about a (fairly extreme) coin toss experiment. Describe your thoughts.

I think it was interesting how through 350,707 tosses they were able to realize about a 1% bias and rather than thinking that with more tosses it would have approached the exact 50%. It is interesting how they investigated this as a bias and connected the concept to physics of how there was a greater chance of it landing on the same side than actually flipping. One thing I would ask is if they truly got rid of the bias in algorithm since not everyone could have truly done a coin flip maybe some people just threw the coin up... anyways it is possible after 350,707 tosses for people to have gotten lazy.

Question 2

In your own words, describe how to take a bootstrap sample.

You have take a sample of some observations maybe in this case heights of a girl friend group to find the average height between a group of girls. Then you can randomly select each height of the girl from a friend group. After choosing every individual data point from the original sample replace it. Repeat the following process n times. Record the mean of the n resampled vales. Then keep going that for like x number of times and use those the x results to calculate the final mean by averaging the other mean values from the intervals of resampling.

Question 3

When do you think we should rely on parametric assumptions (e.g. using a known distribution to approximate something) and when should we rely on simulation-based practices (e.g. something like bootstrap)?

We should use parametric assumptions when we are sure that the data follows a normal distribution or when the sample size is large enough for CLT to apply (so it is more prone to accurately depict a population). We can use simulation-based practice like bootstrapping to compensate for a small sample size and when we don't want to make as many assumptions. Or we can also use simulation-based practices when trying to find medians since it is harder with larger data sets.

Question 4

Describe the importance of EDA before analyzing the data.

EDA helps with summarizing data and catching any errors before running any formal analysis. Without it one can risk making misleading conclusions off incorrect data. EDA can also help analyze trends which is why it is important because it helps with getting the most out of the data and drawing conclusive statements. EDA also helps with analyzing trends even when there are outliers.

Question 5

People also assume that the gender ratio (the ratio of men to women) is 1:1. Skim this article and describe your thoughts.

This really shows a good example of how they are proving correlation is a causation by taking many different factors into aspect of why the gender ratio is not 1:1. Another idea I wanted to note which I do not think was noted, is that in many third-world nations finding out the gender is illegal because at a time many families would have abortion when they found out their baby was female which could also cause the imbalance to the 1:1 ratio. These scientists looking further out from women mortality rates at 10-15 weeks both as a baby and mother, as well as the rate at women being born shows how one conclusion is not always from one investigation but many combined – trends from more than one data distribution.

Coding Questions

For each question below, if you create a variable or an R object, please print out the variable after creating.

Click “Knit” above regularly to generate a pdf of this document. If you only knit the document at the end, it’s much more difficult to troubleshoot.

Section 1: Loading packages

Load the tidyverse library.

```
library(tidyverse)
```

Load the gapminder library.

```
library(gapminder)
```

We will be working with the gapminder dataset, so use `data(...)` to load it into your environment.

```
data("gapminder")
```

Use the `? operator`, and write down what are the columns in the dataset.

The columns are “country”, “continent”, “year”, “gdpPercap”, “lifeExp”, and “pop”.

Use the `head()` function to print out the first 5 rows.

```
gapminder |>
  head(n=5)
```

```
## # A tibble: 5 x 6
##   country    continent year lifeExp      pop gdpPercap
##   <fct>      <fct>    <int>   <dbl>    <int>    <dbl>
## 1 Afghanistan Asia      1952   28.8  8425333    779.
## 2 Afghanistan Asia      1957   30.3  9240934    821.
## 3 Afghanistan Asia      1962   32.0 10267083    853.
## 4 Afghanistan Asia      1967   34.0 11537966    836.
## 5 Afghanistan Asia      1972   36.1 13079460    740.
```

Use the `tail()` function to print out the last 3 rows.

```
gapminder |>
  tail(n=5)
```

```
## # A tibble: 5 x 6
##   country    continent year lifeExp      pop gdpPercap
##   <fct>      <fct>    <int>   <dbl>    <int>    <dbl>
## 1 Zimbabwe Africa      1987   62.4  9216418    706.
## 2 Zimbabwe Africa      1992   60.4 10704340    693.
## 3 Zimbabwe Africa      1997   46.8 11404948    792.
## 4 Zimbabwe Africa      2002   40.0 11926563    672.
## 5 Zimbabwe Africa      2007   43.5 12311143    470.
```

Section 2: dplyr pipelines

TASK 1: Find the average life expectancy and its standard deviation across continents in the year 1997.

First, use `filter()` to create a new dataframe called `gapminder97` with just 1997 data.

```
gapminder97 <- gapminder |>
  filter(year==1997)
gapminder97
```

```
## # A tibble: 142 x 6
##   country    continent year lifeExp      pop gdpPercap
##   <fct>      <fct>    <int>   <dbl>    <int>    <dbl>
## 1 Afghanistan Asia      1997   41.8  22227415    635.
## 2 Albania     Europe      1997   73.0   3428038   3193.
## 3 Algeria     Africa      1997   69.2  29072015   4797.
## 4 Angola      Africa      1997   41.0   9875024   2277.
## 5 Argentina   Americas    1997   73.3  36203463  10967.
## 6 Australia   Oceania     1997   78.8  18565243  26998.
## 7 Austria     Europe      1997   77.5   8069876  29096.
## 8 Bahrain     Asia        1997   73.9   598561   20292.
## 9 Bangladesh  Asia        1997   59.4 123315288    973.
## 10 Belgium    Europe      1997   77.5  10199787  27561.
## # i 132 more rows
```

Then, use `summarize()` to find average life expectancy and its standard deviation for ALL continents.

```
gapminder97 |>
  summarize(mean_lifeExp = mean(lifeExp),
            sd_lifeExp = sd(lifeExp))
```

```
## # A tibble: 1 x 2
##   mean_lifeExp sd_lifeExp
##         <dbl>      <dbl>
## 1      65.0      11.6
```

Add `group_by()` to get your statistics separated by continent. (Be conscious of where this goes in the pipeline!)

```
gapminder |>
  group_by(continent) |>
  summarize(mean_lifeExp = mean(lifeExp),
            sd_lifeExp = sd(lifeExp))
```

```
## # A tibble: 5 x 3
##   continent mean_lifeExp sd_lifeExp
##   <fct>      <dbl>      <dbl>
## 1 Africa      48.9        9.15
## 2 Americas    64.7        9.35
## 3 Asia        60.1       11.9
## 4 Europe      71.9        5.43
## 5 Oceania     74.3        3.80
```

TASK 2: Try to change the code above to get the median population separated by continent AND year.

```
gapminder |>
  group_by(continent, year) |>
  summarize(median_population = median(pop))
```

```
## 'summarise()' has grouped output by 'continent'. You can override using the
## '.groups' argument.
```

```
## # A tibble: 60 x 3
## # Groups:   continent [5]
##   continent year median_population
##   <fct>      <int>      <dbl>
## 1 Africa    1952    2668124.
## 2 Africa    1957    2885790.
## 3 Africa    1962    3145210
## 4 Africa    1967    3473692.
## 5 Africa    1972    3945594.
## 6 Africa    1977    4522666
## 7 Africa    1982    5668228.
## 8 Africa    1987    6635612.
## 9 Africa    1992    7140388.
## 10 Africa   1997    7805422.
## # i 50 more rows
```

TASK 3: Find the Asian continent with the highest GDP per capita in 2002.

First, `filter()` to get only data on Asian countries from 2002. Call this dataframe `Asia2002`.

```
Asia2002 <- gapminder |>
  group_by(continent, year) |>
  filter(continent == "Asia", year == 2002)
```

Then, `arrange()` the data in descending order. (Note that `arrange()` defaults to ascending).

```
Asia2002 |>
  arrange(desc(gdpPercap))
```

```
## # A tibble: 33 x 6
## # Groups:   continent, year [1]
##   country      continent year lifeExp      pop gdpPercap
##   <fct>         <fct>    <int>  <dbl>    <int>    <dbl>
## 1 Singapore      Asia      2002   78.8  4197776  36023.
## 2 Kuwait          Asia      2002   76.9  2111561  35110.
## 3 Hong Kong, China Asia      2002   81.5  6762476  30209.
## 4 Japan           Asia      2002    82 127065841  28605.
## 5 Bahrain         Asia      2002   74.8   656397  23404.
## 6 Taiwan          Asia      2002   77.0 22454239  23235.
## 7 Israel          Asia      2002   79.7  6029529  21906.
## 8 Oman            Asia      2002   74.2  2713462  19775.
## 9 Korea, Rep.     Asia      2002   77.0 47969150  19234.
## 10 Saudi Arabia   Asia      2002   71.6 24501530  19015.
## # i 23 more rows
```

Use `select()` to only show the country, continent, year and gdp per capita columns.

```
Asia2002 |>
  arrange(desc(gdpPercap)) |>
  select(country, continent, year, gdpPercap)
```

```
## # A tibble: 33 x 4
## # Groups:   continent, year [1]
##   country      continent year gdpPercap
##   <fct>         <fct>    <int>    <dbl>
## 1 Singapore      Asia      2002  36023.
## 2 Kuwait          Asia      2002  35110.
## 3 Hong Kong, China Asia      2002  30209.
## 4 Japan           Asia      2002  28605.
## 5 Bahrain         Asia      2002  23404.
## 6 Taiwan          Asia      2002  23235.
## 7 Israel          Asia      2002  21906.
## 8 Oman            Asia      2002  19775.
## 9 Korea, Rep.     Asia      2002  19234.
## 10 Saudi Arabia   Asia      2002  19015.
## # i 23 more rows
```

Use `head()` to print only the first row.

```
Asia2002 |>
  arrange(desc(gdpPercap)) |>
  select(country, continent, year, gdpPercap) |>
  head(n=1)
```

```
## # A tibble: 1 x 4
## # Groups:   continent, year [1]
##   country continent year gdpPercap
##   <fct>    <fct>    <int>    <dbl>
## 1 Singapore Asia      2002    36023.
```

TASK 4: Find the country with the highest real GDP in 1972 in Europe.

Note that $GDPPerCapita = \frac{RealGDP}{Population}$. Use `mutate()` to create a `realGDP` column.

```
Europe72 <- gapminder |>
  filter(year == 1972,
         continent == "Europe") |>
  mutate(realGDP = gdpPercap * pop)
```

Repeat similar steps from Task 3.

```
Europe72 |>
  arrange(desc(realGDP)) |>
  select(country, continent, year, gdpPercap, realGDP) |>
  head(n=1)
```

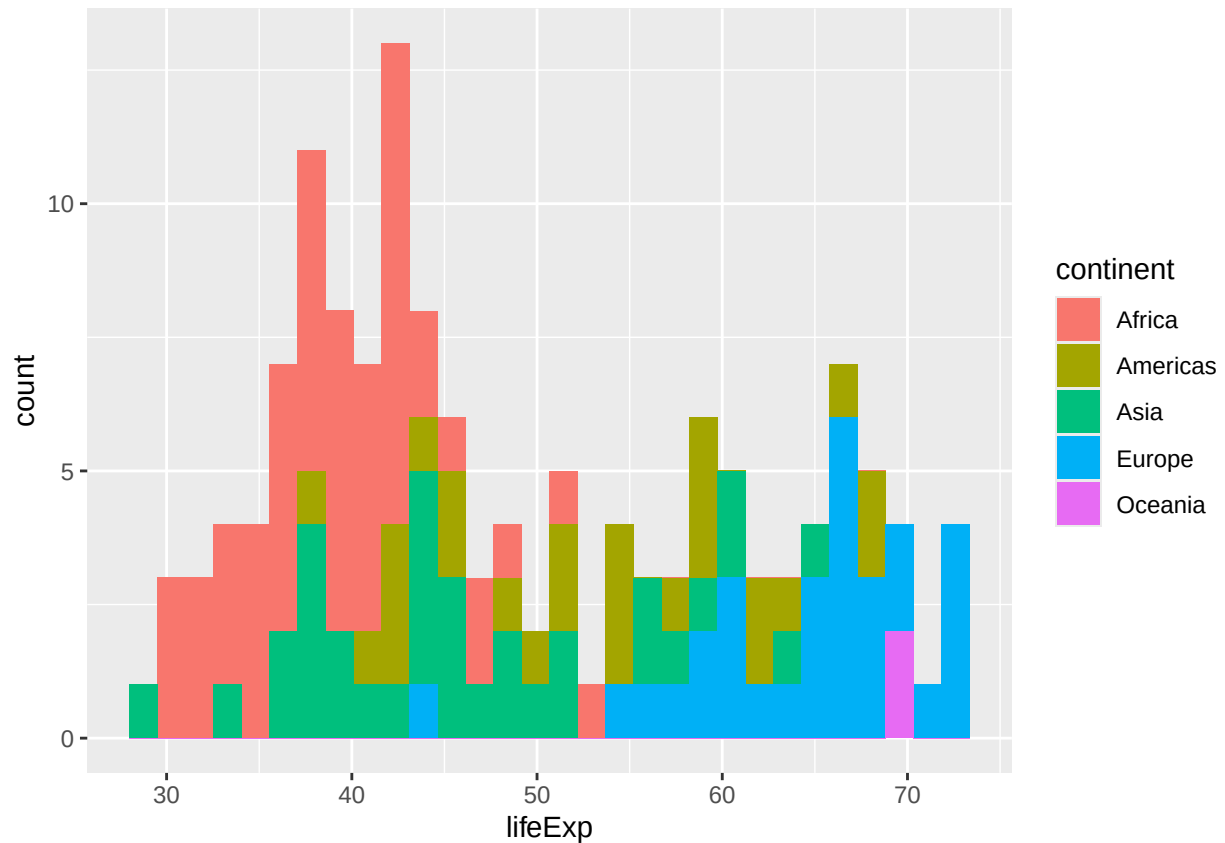
```
## # A tibble: 1 x 5
##   country continent year gdpPercap realGDP
##   <fct>    <fct>    <int>    <dbl>    <dbl>
## 1 Germany Europe      1972    18016.  1.42e12
```

Section 3 : ggplot()

TASK 1: Create a histogram for life expectancy in 1952, with color representing the continent.

```
gapminder |>
  filter(year == 1952) |>
  ggplot(aes(x=lifeExp, fill = continent))+
  geom_histogram()
```

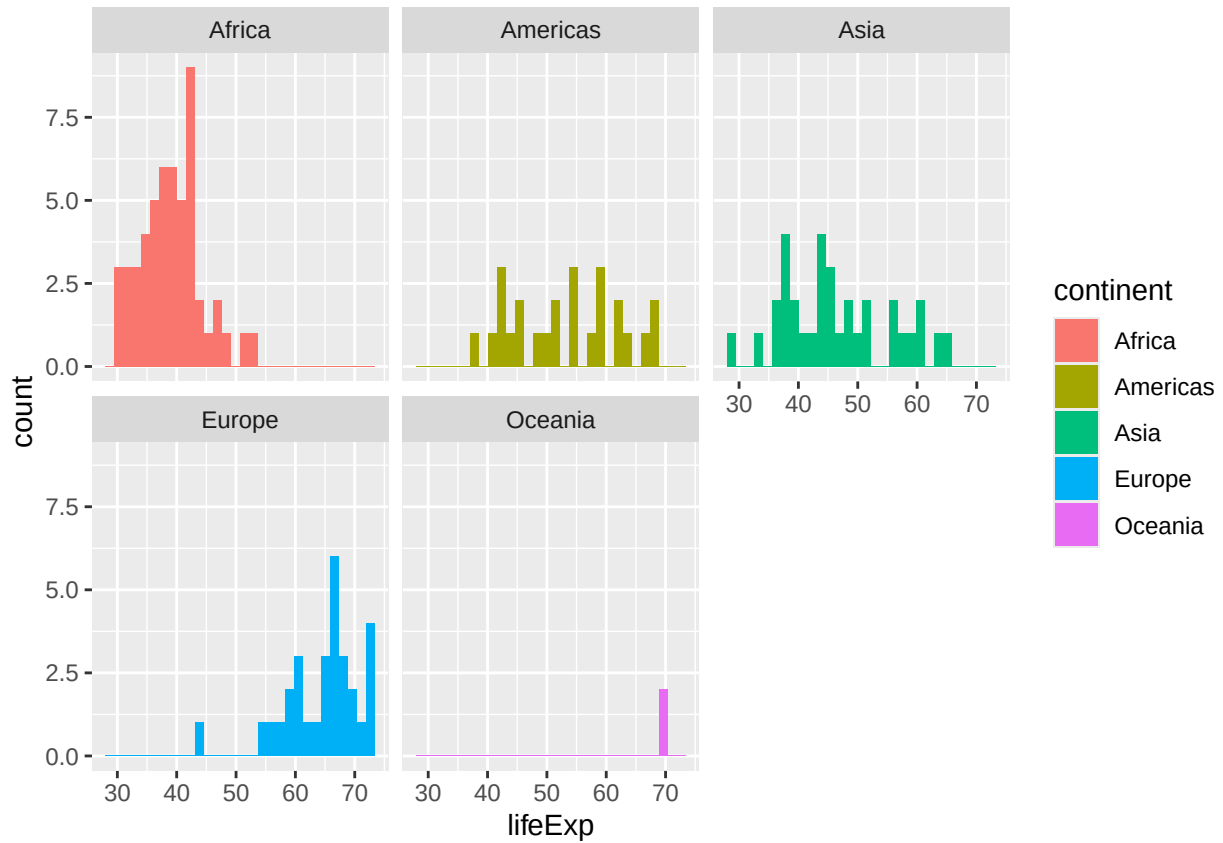
```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```



TASK 2: Create a histogram for life expectancy in 1952, faceted by continent using `facet_wrap()`. Which plot do you prefer: Task 1 or Task 2? Why?

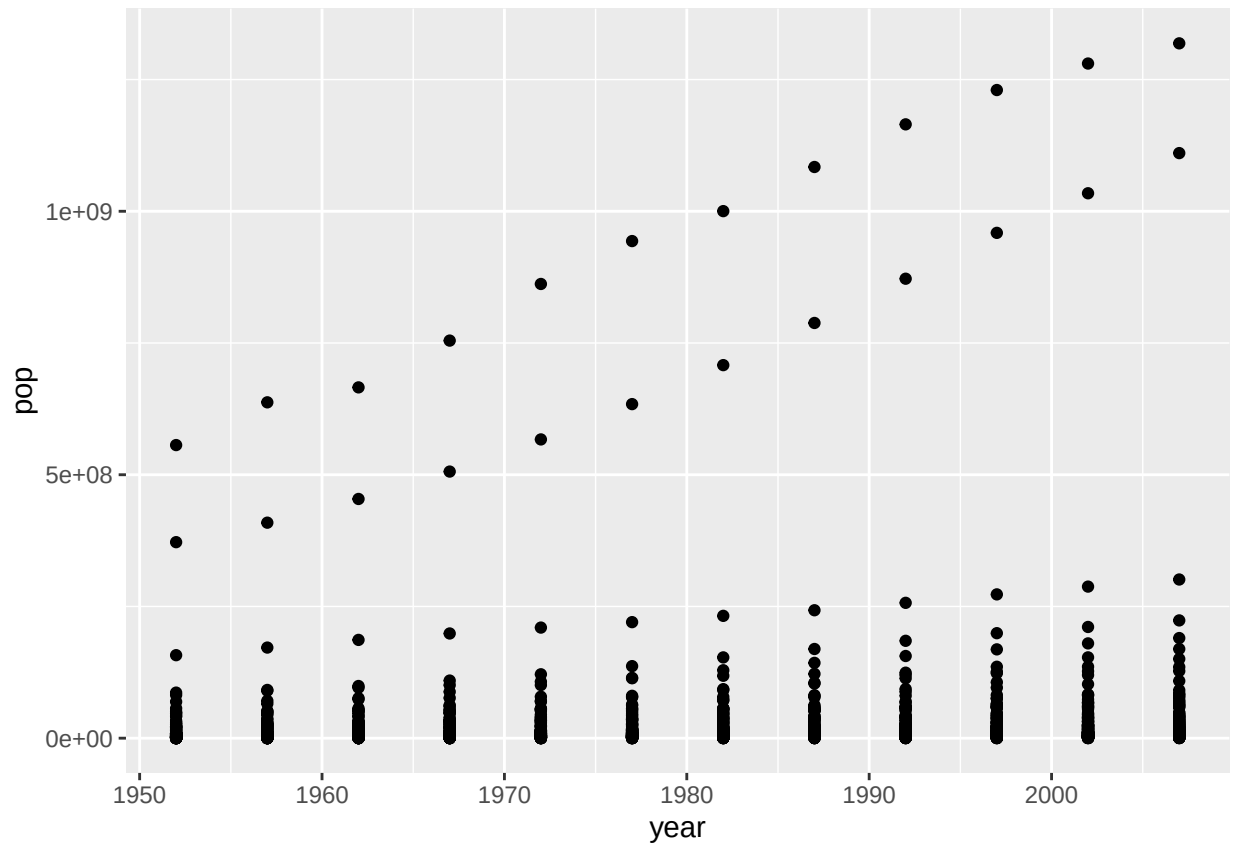
```
gapminder |>
  filter(year == 1952) |>
  ggplot(aes(x=lifeExp, fill = continent))+
  geom_histogram()+
  facet_wrap(vars(continent))
```

'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.



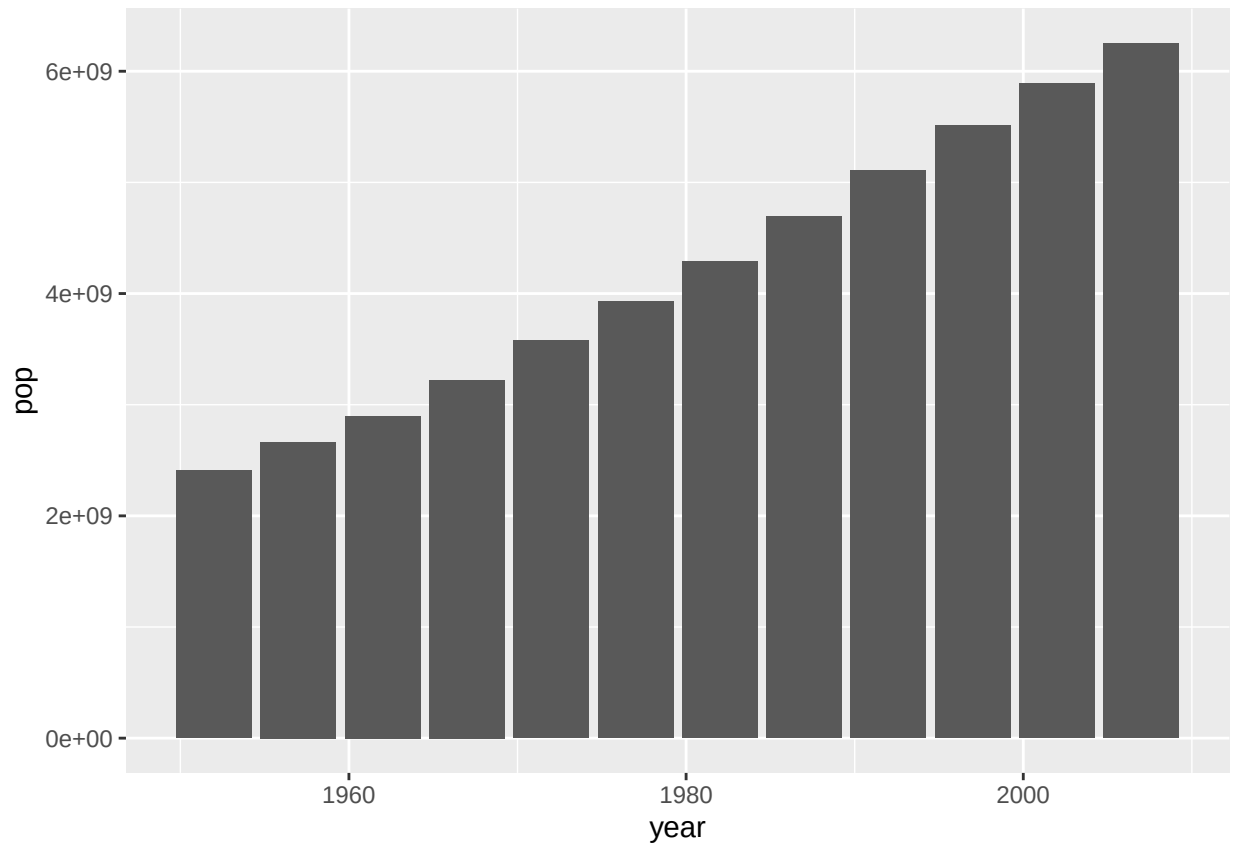
TASK 3: Create a scatterplot of year vs. population for a country of your choosing.

```
gapminder |>
  ggplot(aes(x=year, y=pop))+
  geom_point()
```

TASK 4: Create a bar chart of year vs. population for the same country using `geom_col()`. Which plot do you prefer: Task 3 or Task 4? Why?

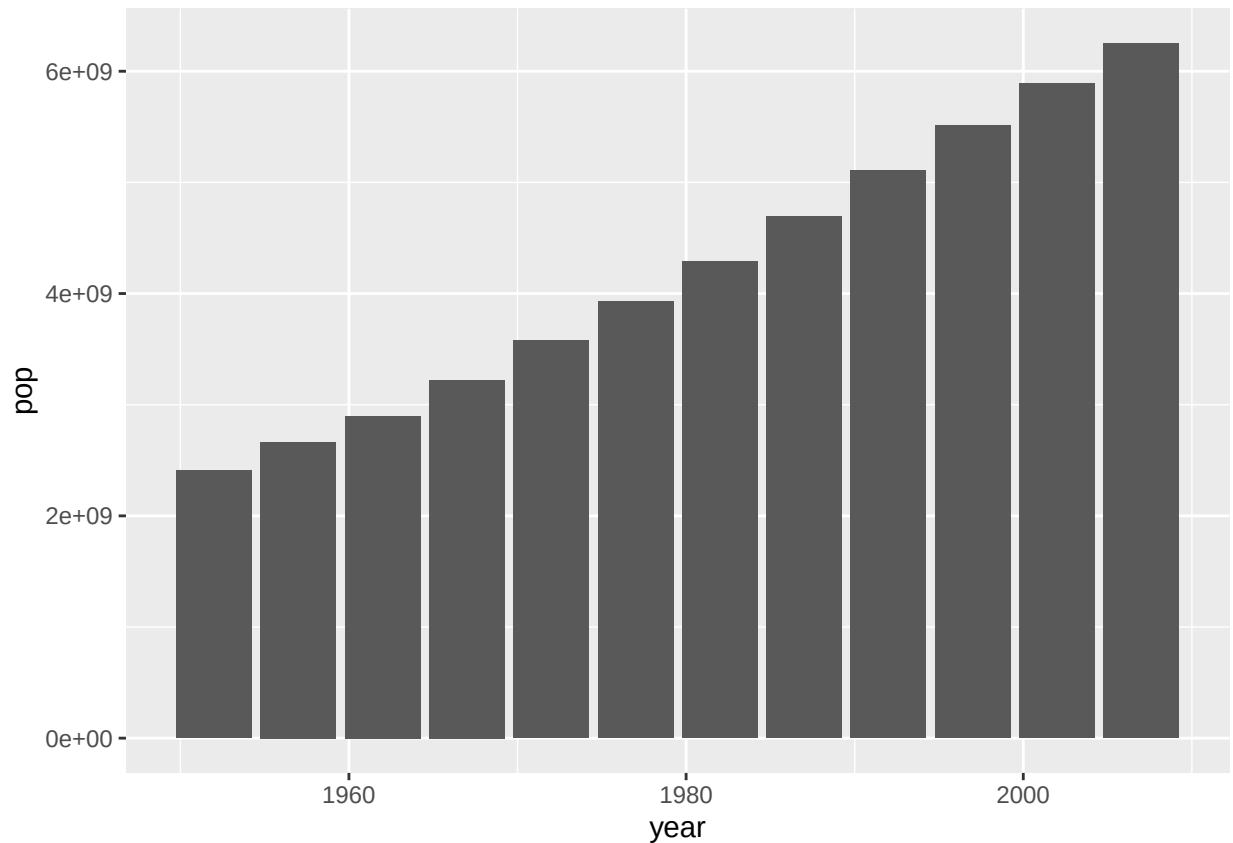
```
gapminder |>
  ggplot(aes(x=year, y=pop))+
  geom_col()
```



Written answer here.

TASK 5: Using your plot from Task 4, add a `geom_line()`.

```
gapminder |>  
  ggplot(aes(x=year, y=pop))+  
  geom_col()
```

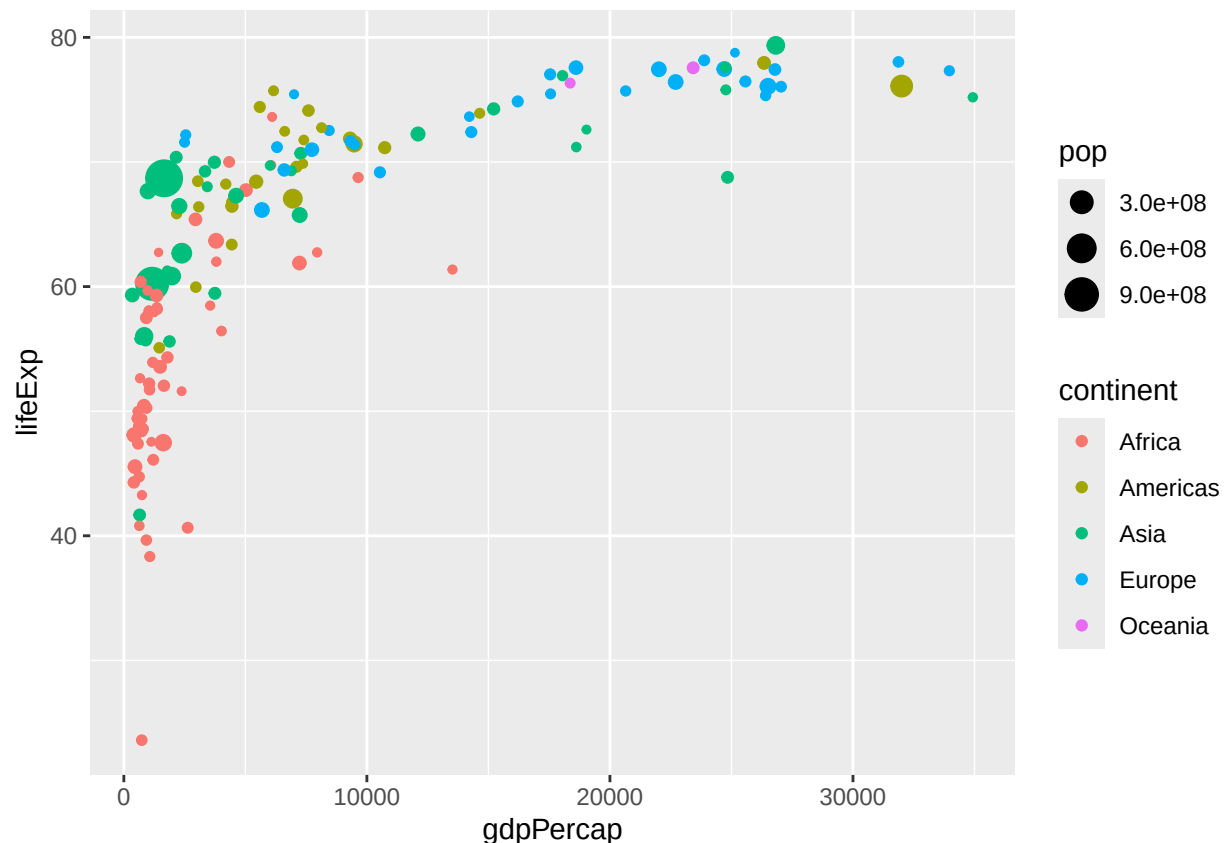


```
geom_line()
```

```
## geom_line: na.rm = FALSE, orientation = NA
## stat_identity: na.rm = FALSE
## position_identity
```

TASK 6: Recreate the Hans Rosling plot for the year 1992 (no animations!).

```
gapminder |>
  filter(year == 1992) |>
  ggplot(aes(x=gdpPercap,
             y=lifeExp,
             size = pop,
             color = continent)) +
  geom_point()
```



Section 4: PDFs and CDF

Recall the game from your first quiz: Consider the following game. Roll a fair 8-sided die. If you roll an even number, you lose \$5. If you roll 5 or 7, you win \$1. If you roll a 3, you get nothing. If you roll a 1, you win \$10. What's the expected value of this game? Show your work.

TASK 1: Create a dataframe called `pdf` that has two columns: one called `x` for the outcomes and one called `prob` for the respective probabilities. Use this dataframe and `dplyr` functions to calculate expected value.

First, create the dataframe:

```
pdf <- data.frame(x = c(-5,1,0,10),
                  prob = c(0.5, 0.25, 0.125, 0.125))
```

Then, use `mutate()` and `summarize()` to calculate the expected value.

```
pdf|>
  mutate(outcome = x*prob) |>
  summarize(weighted_mean = sum(outcome))
```

```
## weighted_mean
## 1             -1
```

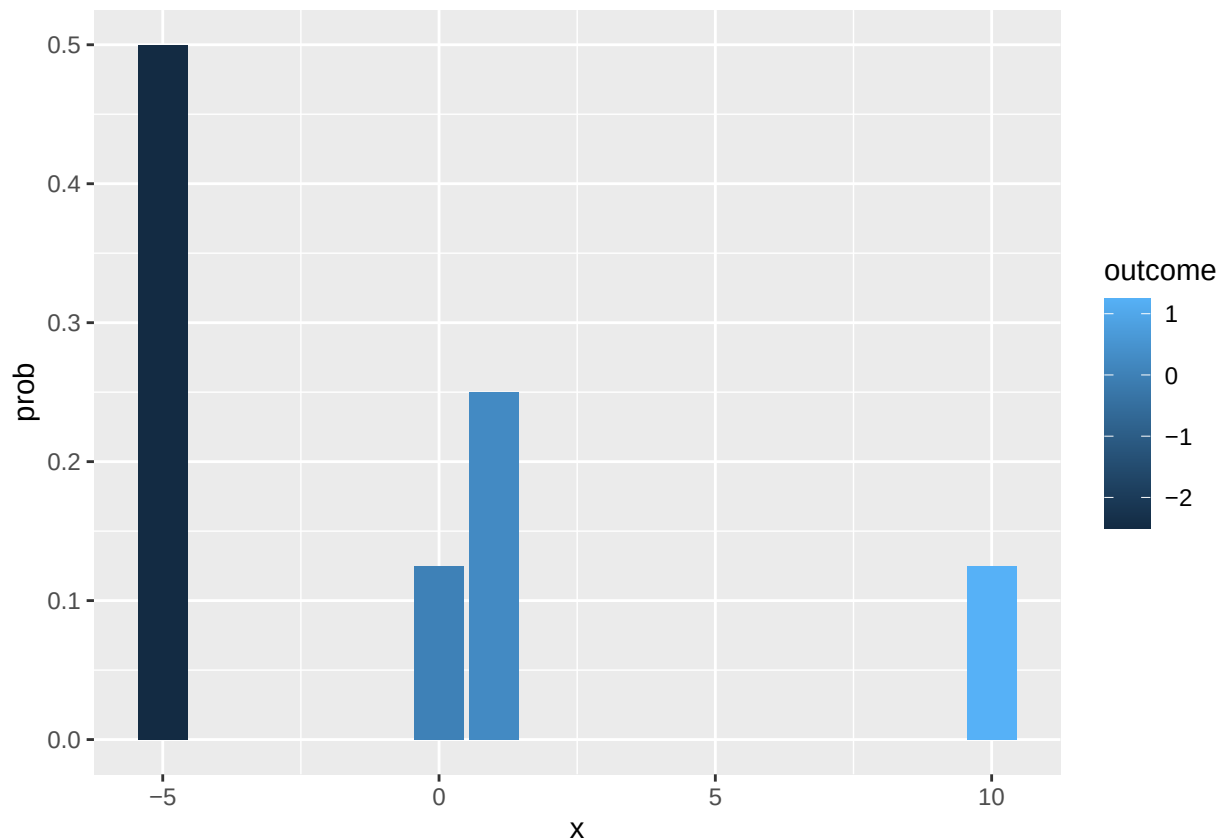
Use `weighted.mean()` to check your answer.

```
weighted.mean(pdf$x, pdf$prob)
```

```
## [1] -1
```

Use `ggplot` to visualize the probability distribution.

```
pdf |>
  mutate(outcome = x*prob) |>
  ggplot(aes(x=x, y=prob, fill=outcome))+
  geom_col()
```



TASK 2: Find the cdf of this distribution.

Hint: use `mutate()` and the function `cumsum()` .

```
cdf <- pdf |>
  mutate(prob = cumsum(prob))
cdf
```

```
##   x  prob
## 1 -5 0.500
## 2  1 0.750
## 3  0 0.875
## 4 10 1.000
```

Section 5: Probability Simulations

TASK 1: Women's height tend to be normally distributed. Let's plot it!

First, find the average and standard deviation of American women (in inches) and save it as two variables.

```
women_ht_avg <- 63.6
women_ht_sd <- 2.5
```

Then, create a dataframe of x-values (from 50 to 75, at 0.1 intervals).

```
x = (seq(from=50, to=75, by=0.1))
df = data.frame(x)
df
```

Use `mutate()` to calculate the probabilities of those x-values.

```
df <- data.frame(x = x) |>
  mutate(probability = dnorm(x, mean = women_ht_avg, sd = women_ht_sd))
df
```

```
##      x probability
## 1  50.0 5.981381e-08
## 2  50.1 7.429447e-08
## 3  50.2 9.213331e-08
## 4  50.3 1.140728e-07
## 5  50.4 1.410108e-07
## 6  50.5 1.740315e-07
## 7  50.6 2.144414e-07
## 8  50.7 2.638120e-07
## 9  50.8 3.240302e-07
## 10 50.9 3.973576e-07
## 11 51.0 4.864998e-07
## 12 51.1 5.946878e-07
## 13 51.2 7.257725e-07
## 14 51.3 8.843355e-07
## 15 51.4 1.075818e-06
## 16 51.5 1.306669e-06
## 17 51.6 1.584520e-06
## 18 51.7 1.918380e-06
## 19 51.8 2.318873e-06
## 20 51.9 2.798494e-06
## 21 52.0 3.371917e-06
## 22 52.1 4.056341e-06
## 23 52.2 4.871887e-06
## 24 52.3 5.842047e-06
## 25 52.4 6.994200e-06
## 26 52.5 8.360191e-06
## 27 52.6 9.976989e-06
## 28 52.7 1.188743e-05
## 29 52.8 1.414104e-05
## 30 52.9 1.679500e-05
```

```
## 31 53.0 1.991516e-05
## 32 53.1 2.357723e-05
## 33 53.2 2.786806e-05
## 34 53.3 3.288713e-05
## 35 53.4 3.874808e-05
## 36 53.5 4.558056e-05
## 37 53.6 5.353209e-05
## 38 53.7 6.277025e-05
## 39 53.8 7.348500e-05
## 40 53.9 8.589119e-05
## 41 54.0 1.002314e-04
## 42 54.1 1.167788e-04
## 43 54.2 1.358405e-04
## 44 54.3 1.577610e-04
## 45 54.4 1.829259e-04
## 46 54.5 2.117659e-04
## 47 54.6 2.447608e-04
## 48 54.7 2.824443e-04
## 49 54.8 3.254085e-04
## 50 54.9 3.743089e-04
## 51 55.0 4.298693e-04
## 52 55.1 4.928877e-04
## 53 55.2 5.642409e-04
## 54 55.3 6.448910e-04
## 55 55.4 7.358905e-04
## 56 55.5 8.383882e-04
## 57 55.6 9.536353e-04
## 58 55.7 1.082990e-03
## 59 55.8 1.227925e-03
## 60 55.9 1.390031e-03
## 61 56.0 1.571021e-03
## 62 56.1 1.772739e-03
## 63 56.2 1.997160e-03
## 64 56.3 2.246393e-03
## 65 56.4 2.522691e-03
## 66 56.5 2.828442e-03
## 67 56.6 3.166181e-03
## 68 56.7 3.538582e-03
## 69 56.8 3.948462e-03
## 70 56.9 4.398775e-03
## 71 57.0 4.892611e-03
## 72 57.1 5.433188e-03
## 73 57.2 6.023847e-03
## 74 57.3 6.668040e-03
## 75 57.4 7.369324e-03
## 76 57.5 8.131342e-03
## 77 57.6 8.957812e-03
## 78 57.7 9.852508e-03
## 79 57.8 1.081924e-02
## 80 57.9 1.186183e-02
## 81 58.0 1.298411e-02
## 82 58.1 1.418984e-02
## 83 58.2 1.548274e-02
## 84 58.3 1.686644e-02
```

85 58.4 1.834443e-02
86 58.5 1.992004e-02
87 58.6 2.159639e-02
88 58.7 2.337638e-02
89 58.8 2.526262e-02
90 58.9 2.725743e-02
91 59.0 2.936273e-02
92 59.1 3.158006e-02
93 59.2 3.391054e-02
94 59.3 3.635479e-02
95 59.4 3.891291e-02
96 59.5 4.158444e-02
97 59.6 4.436833e-02
98 59.7 4.726292e-02
99 59.8 5.026585e-02
100 59.9 5.337412e-02
101 60.0 5.658399e-02
102 60.1 5.989099e-02
103 60.2 6.328992e-02
104 60.3 6.677482e-02
105 60.4 7.033897e-02
106 60.5 7.397491e-02
107 60.6 7.767442e-02
108 60.7 8.142856e-02
109 60.8 8.522766e-02
110 60.9 8.906140e-02
111 61.0 9.291880e-02
112 61.1 9.678829e-02
113 61.2 1.006577e-01
114 61.3 1.045145e-01
115 61.4 1.083456e-01
116 61.5 1.121375e-01
117 61.6 1.158766e-01
118 61.7 1.195490e-01
119 61.8 1.231405e-01
120 61.9 1.266372e-01
121 62.0 1.300249e-01
122 62.1 1.332898e-01
123 62.2 1.364183e-01
124 62.3 1.393970e-01
125 62.4 1.422130e-01
126 62.5 1.448540e-01
127 62.6 1.473081e-01
128 62.7 1.495642e-01
129 62.8 1.516122e-01
130 62.9 1.534425e-01
131 63.0 1.550466e-01
132 63.1 1.564171e-01
133 63.2 1.575473e-01
134 63.3 1.584321e-01
135 63.4 1.590671e-01
136 63.5 1.594493e-01
137 63.6 1.595769e-01
138 63.7 1.594493e-01

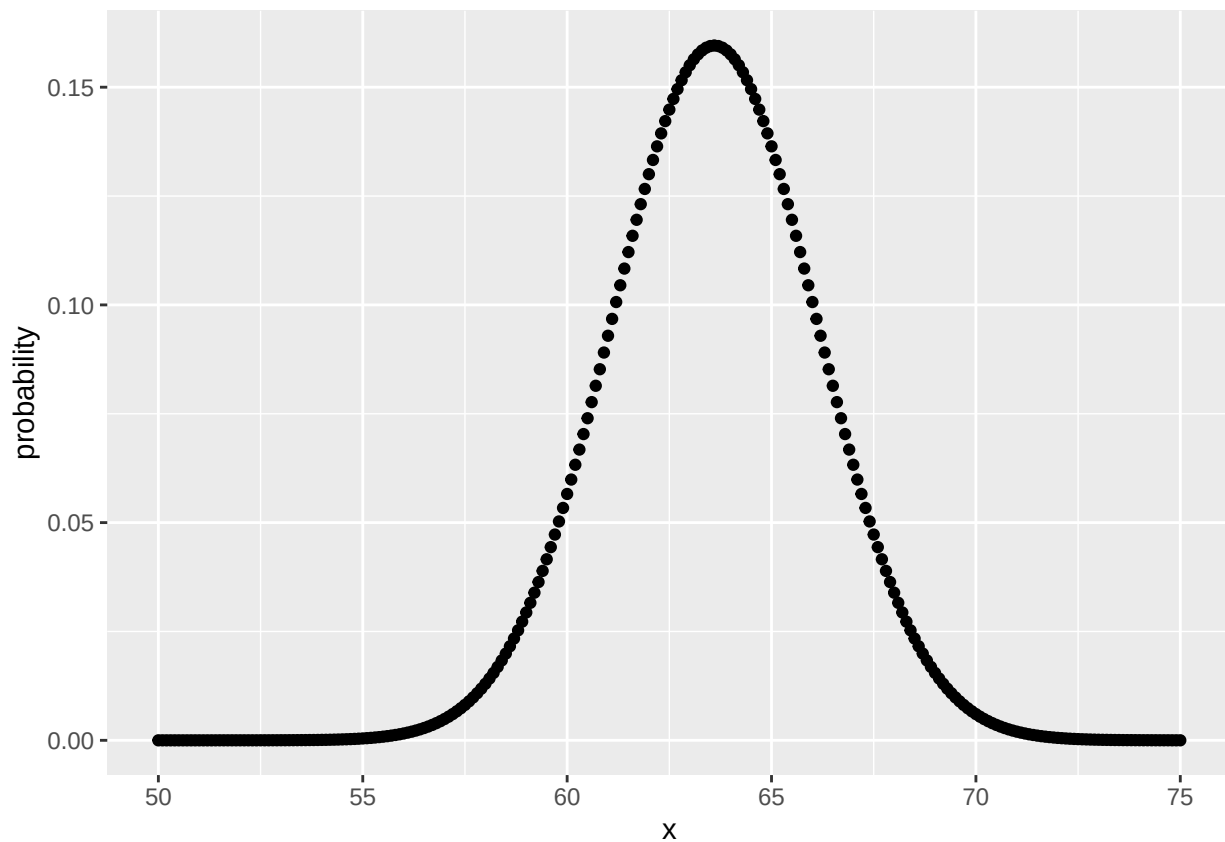
139 63.8 1.590671e-01
140 63.9 1.584321e-01
141 64.0 1.575473e-01
142 64.1 1.564171e-01
143 64.2 1.550466e-01
144 64.3 1.534425e-01
145 64.4 1.516122e-01
146 64.5 1.495642e-01
147 64.6 1.473081e-01
148 64.7 1.448540e-01
149 64.8 1.422130e-01
150 64.9 1.393970e-01
151 65.0 1.364183e-01
152 65.1 1.332898e-01
153 65.2 1.300249e-01
154 65.3 1.266372e-01
155 65.4 1.231405e-01
156 65.5 1.195490e-01
157 65.6 1.158766e-01
158 65.7 1.121375e-01
159 65.8 1.083456e-01
160 65.9 1.045145e-01
161 66.0 1.006577e-01
162 66.1 9.678829e-02
163 66.2 9.291880e-02
164 66.3 8.906140e-02
165 66.4 8.522766e-02
166 66.5 8.142856e-02
167 66.6 7.767442e-02
168 66.7 7.397491e-02
169 66.8 7.033897e-02
170 66.9 6.677482e-02
171 67.0 6.328992e-02
172 67.1 5.989099e-02
173 67.2 5.658399e-02
174 67.3 5.337412e-02
175 67.4 5.026585e-02
176 67.5 4.726292e-02
177 67.6 4.436833e-02
178 67.7 4.158444e-02
179 67.8 3.891291e-02
180 67.9 3.635479e-02
181 68.0 3.391054e-02
182 68.1 3.158006e-02
183 68.2 2.936273e-02
184 68.3 2.725743e-02
185 68.4 2.526262e-02
186 68.5 2.337638e-02
187 68.6 2.159639e-02
188 68.7 1.992004e-02
189 68.8 1.834443e-02
190 68.9 1.686644e-02
191 69.0 1.548274e-02
192 69.1 1.418984e-02

193 69.2 1.298411e-02
194 69.3 1.186183e-02
195 69.4 1.081924e-02
196 69.5 9.852508e-03
197 69.6 8.957812e-03
198 69.7 8.131342e-03
199 69.8 7.369324e-03
200 69.9 6.668040e-03
201 70.0 6.023847e-03
202 70.1 5.433188e-03
203 70.2 4.892611e-03
204 70.3 4.398775e-03
205 70.4 3.948462e-03
206 70.5 3.538582e-03
207 70.6 3.166181e-03
208 70.7 2.828442e-03
209 70.8 2.522691e-03
210 70.9 2.246393e-03
211 71.0 1.997160e-03
212 71.1 1.772739e-03
213 71.2 1.571021e-03
214 71.3 1.390031e-03
215 71.4 1.227925e-03
216 71.5 1.082990e-03
217 71.6 9.536353e-04
218 71.7 8.383882e-04
219 71.8 7.358905e-04
220 71.9 6.448910e-04
221 72.0 5.642409e-04
222 72.1 4.928877e-04
223 72.2 4.298693e-04
224 72.3 3.743089e-04
225 72.4 3.254085e-04
226 72.5 2.824443e-04
227 72.6 2.447608e-04
228 72.7 2.117659e-04
229 72.8 1.829259e-04
230 72.9 1.577610e-04
231 73.0 1.358405e-04
232 73.1 1.167788e-04
233 73.2 1.002314e-04
234 73.3 8.589119e-05
235 73.4 7.348500e-05
236 73.5 6.277025e-05
237 73.6 5.353209e-05
238 73.7 4.558056e-05
239 73.8 3.874808e-05
240 73.9 3.288713e-05
241 74.0 2.786806e-05
242 74.1 2.357723e-05
243 74.2 1.991516e-05
244 74.3 1.679500e-05
245 74.4 1.414104e-05
246 74.5 1.188743e-05

```
## 247 74.6 9.976989e-06
## 248 74.7 8.360191e-06
## 249 74.8 6.994200e-06
## 250 74.9 5.842047e-06
## 251 75.0 4.871887e-06
```

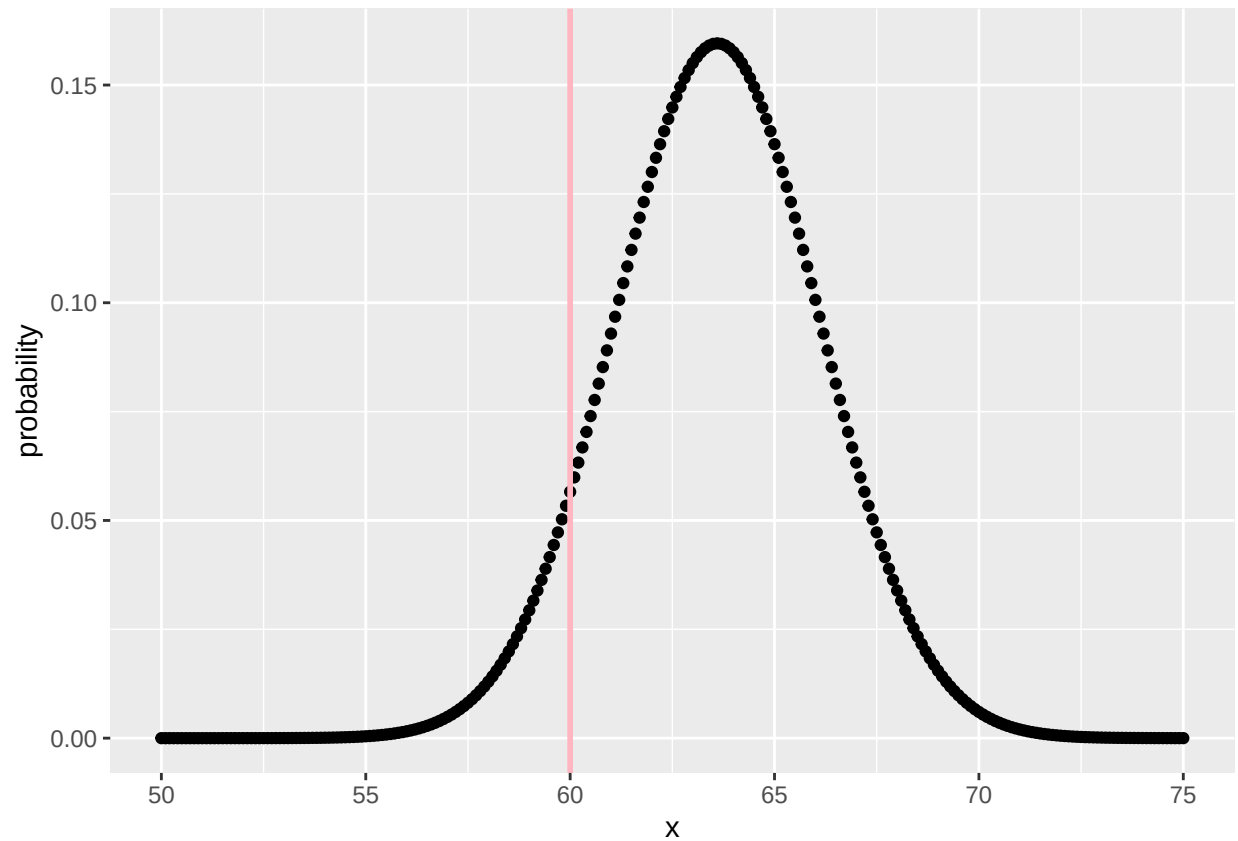
Plot these points (using a scatterplot).

```
df |>
  ggplot(aes(x=x, y=probability))+
  geom_point()
```



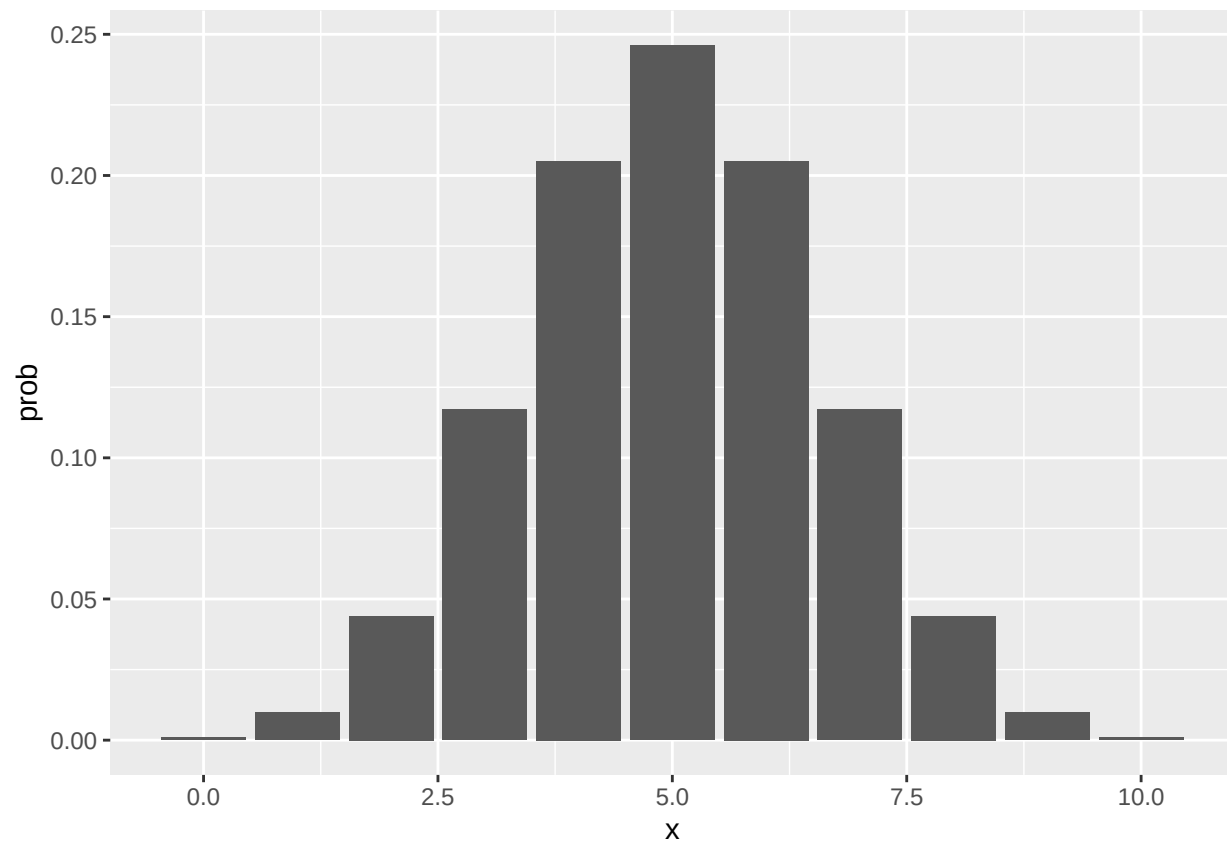
Add a vertical line at your own height on the plot.

```
df |>
  ggplot(aes(x=x, y=probability))+
  geom_point()+
  geom_vline(xintercept = 60, color = "lightpink", linewidth =1)
```



TASK 2: Create a plot of the pdf for 10 rolls of a die with success defined as rolling a prime number.

```
x <- 0:10
data.frame(x=x) |>
  mutate(prob=dbinom(x, size=10, prob=0.5)) |>
  ggplot(aes(x=x, y=prob))+
  geom_col()
```



TASK 3: Are you more likely to get 5 heads in 10 tosses of a fair coin or 50 heads in 100 tosses of a fair coin?

```
toss1 = dbinom(5, size=10, prob=0.5)
toss2 = dbinom(50, size=100, prob=0.5)
toss1
```

```
## [1] 0.2460938
```

```
toss2
```

```
## [1] 0.07958924
```

TASK 4: What's the probability of getting at most 50 heads in 100 tosses of a fair coin?

```
pbinom(q=50, size=100, prob=0.5)
```

```
## [1] 0.5397946
```

```
sum(dbinom(x=0:50, size=100, prob=0.5))
```

```
## [1] 0.5397946
```

TASK 5: Create a plot of the pdf for a Poisson distribution with $\lambda = 1$. Use `geom_point` AND `geom_line`.

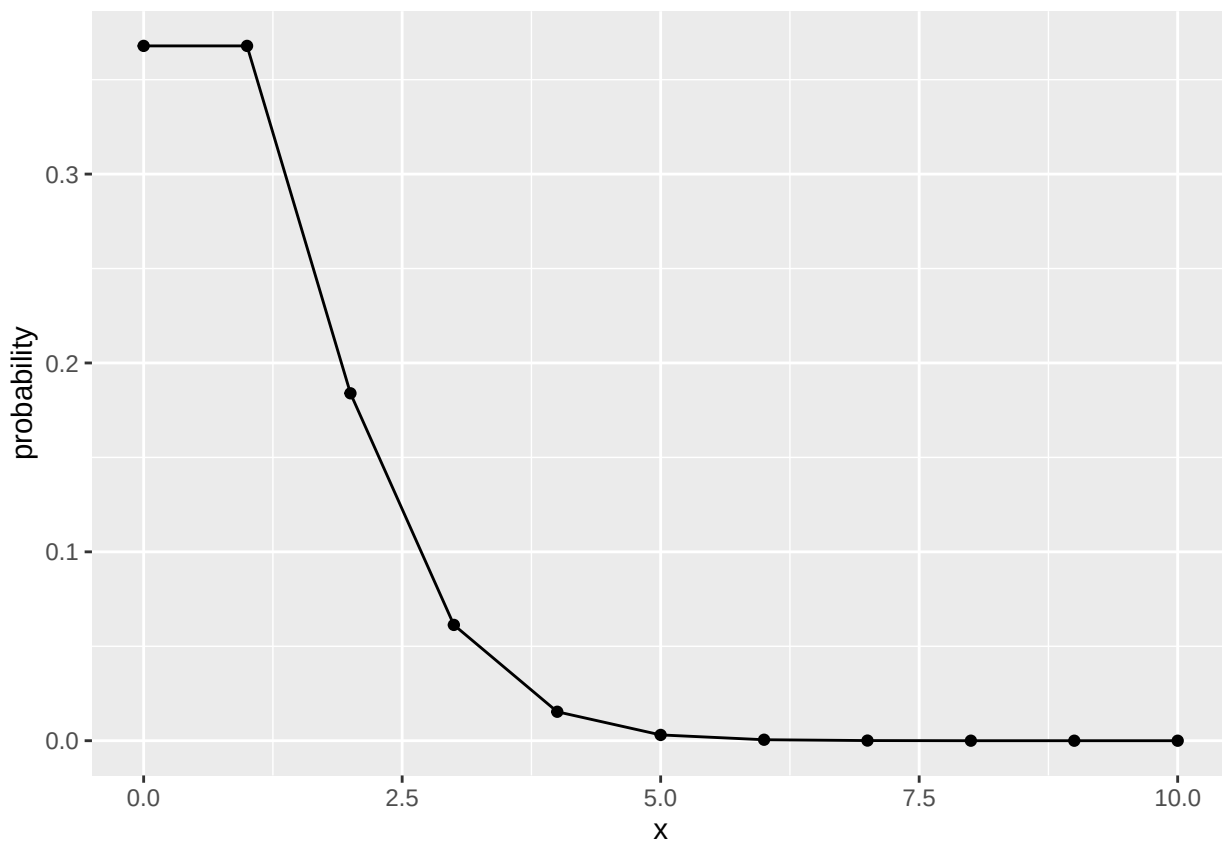
Hint: use integers from 0 to 10 as your x-values

Written answer here

```
lambda_val <- 1
x_vals_plot <- 0:10

pdf_data <- data.frame(
  x = x_vals_plot,
  probability = dpois(x_vals_plot, lambda = lambda_val)
)

ggplot(pdf_data, aes(x = x, y = probability)) +
  geom_point() +
  geom_line()
```

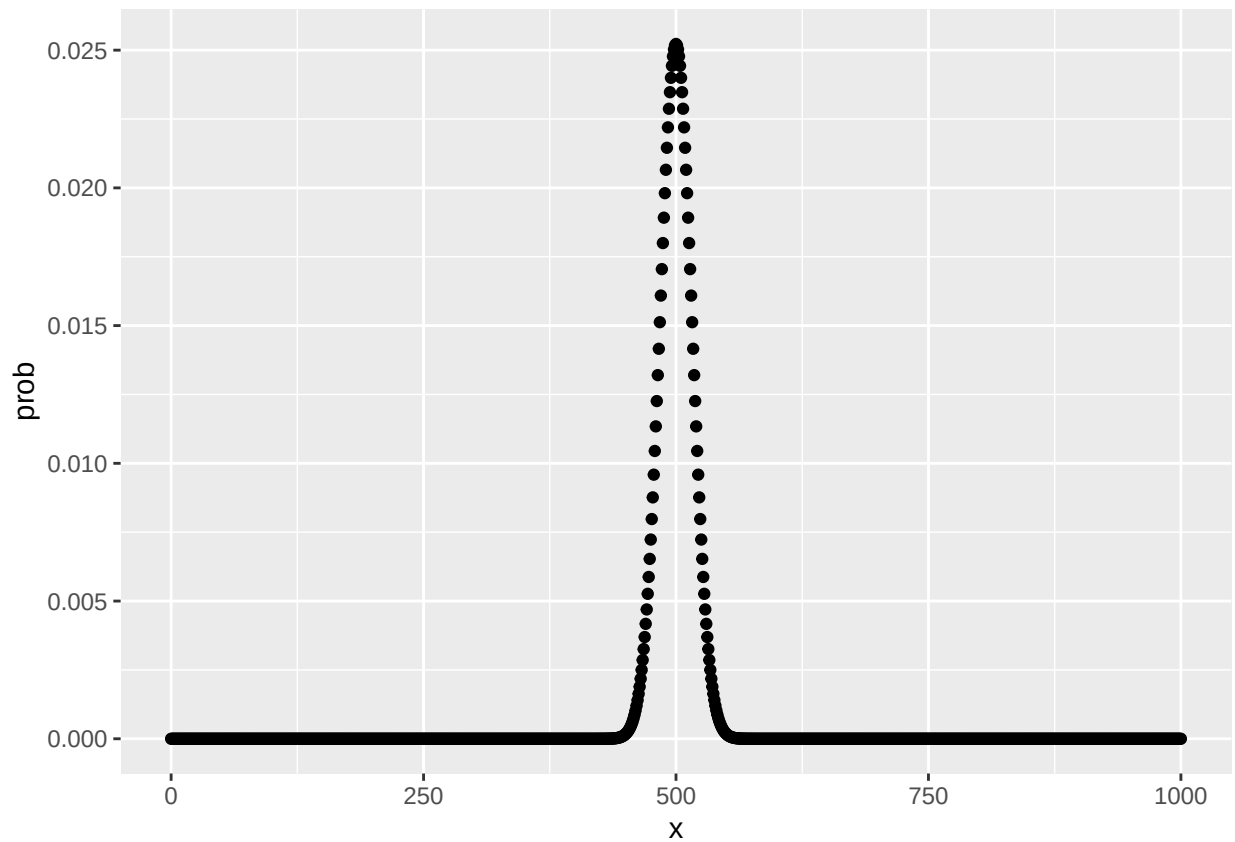


Section 6: Comparing Distributions

TASK 1: Assuming a large number of trials ($n = 100+$), for what values of p does the binomial distribution look very similar to a normal distribution? Use plots of the pdf to demonstrate.

Written answer here.

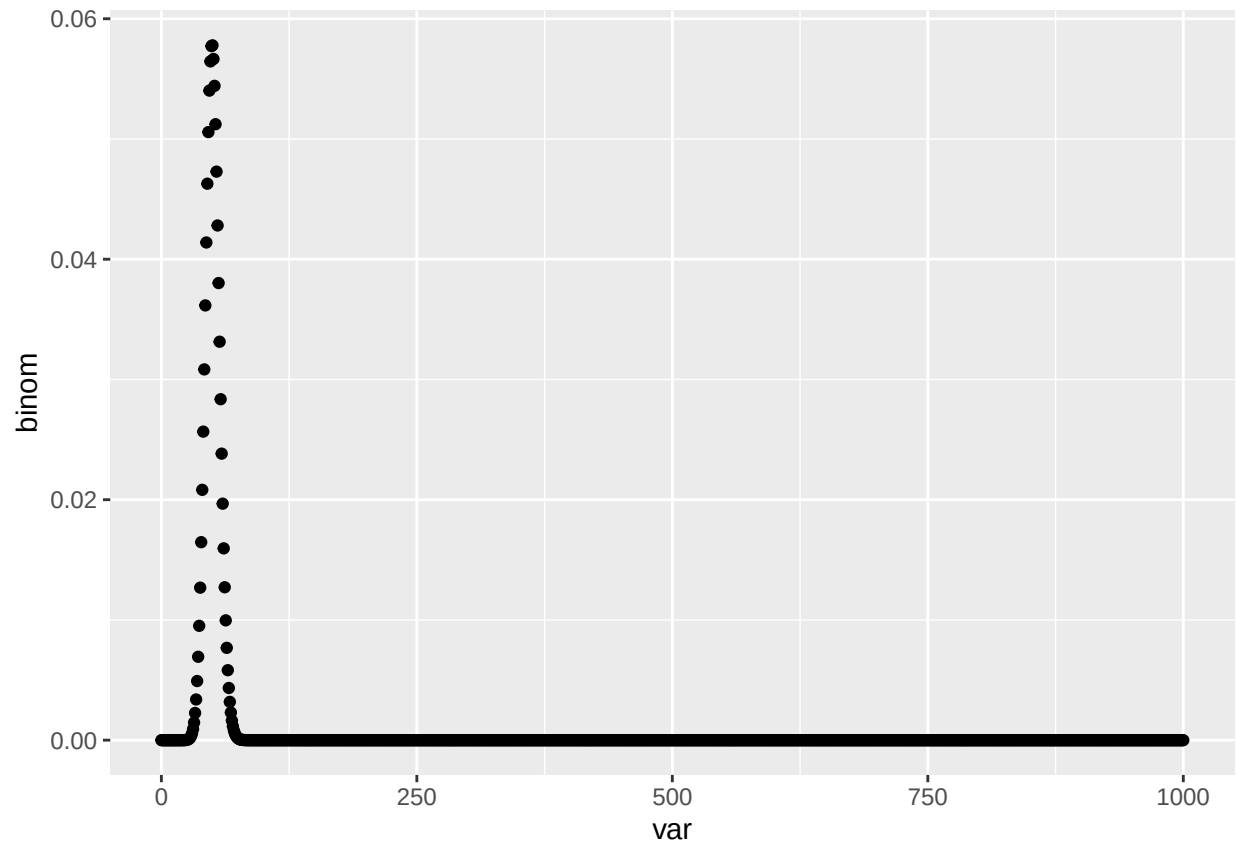
```
x <- 0:1000
data.frame(x=x) |>
  mutate(prob = dbinom(x,size=1000,prob=0.5)) |>
  ggplot(aes(x=x, y=prob))+
  geom_point()
```



TASK 2: Assuming a large number of trials ($n = 100+$), for what values of p does the binomial distribution look very similar to the Poisson distribution above? Use plots of the pdf to demonstrate.

Written answer here.

```
data.frame(
var <- c(0:1000),
  binom <- dbinom(var,1000,0.05)) |>
  ggplot(aes(x=var,
              y=binom))+
  geom_point()
```



TASK 3: For what values of lambda does a Poisson distribution look very similar to a normal distribution. Use plots of the pdf to demonstrate.

Written answer here.

```
x = 0:1000
probab <- dpois(x,
                 lambda = 500)
data_poisson <- data.frame(x=x,
                           y=probab)

data_poisson |>
ggplot(aes(x=x,
           y= probab))+
  geom_point() + geom_line()
```